AUTOML – MLOPS – PLATFORM – PROJECT GUIDE

Steps:

1. **Make Folder name of:** AutoMl-Mlops-Platform
2. **Make virtual Environment:**
   - Python -m venv venv
   - venv\scripts\activate
3. **Make and Run requirements.txt file:**
   - pip install -r requirements.txt
   - pip install -r requirements.txt --prefer-binary

Then verify:

- python -c "import pandas, sklearn, mlflow, fastapi; print('All good!')"

4. **IF requirements.txt not install requirements then we have other option for same operation**:
   - pip install pandas numpy joblib scikit-learn xgboost mlflow fastapi uvicorn pydantic python-multipart pyyaml python-dotenv pytest httpx prometheus-client dvc --prefer-binary

5. **Make all Folders and Files on main folder by using one command:**

   - @("src\data_ingestion\ingest.py", "src\data_validation\validate.py", "src\feature_engineering\features.py", "src\model_training\train.py", "src\model_evaluation\evaluate.py", "src\deployment\api.py", "pipeline\training_pipeline.py", "config\config.yaml", "params.yaml", "dvc.yaml", "Dockerfile", ".github\workflows\ci.yaml", "README.md") | ForEach-Object { New-Item -ItemType File -Path $_ -Force }

6. **Then verify everything was created:**

   You should see your full project structure like this:

```
AUTOML-MLOPS-PLATFORM
├── data
│   ├── raw
│   └── processed
├── src
│   ├── data_ingestion
│   │   └── ingest.py
│   ├── data_validation
│   │   └── validate.py
│   ├── feature_engineering
│   │   └── features.py
│   ├── model_training
│   │   └── train.py
│   ├── model_evaluation
│   │   └── evaluate.py
│   └── deployment
│       └── api.py
├── pipeline
│   └── training_pipeline.py
├── config
│   └── config.yaml
├── artifacts
├── tests
├── notebooks
├── .github
│   └── workflows
│       └── ci.yaml
├── params.yaml
├── dvc.yaml
├── Dockerfile
└── README.md
```

7. **Initialize Git:**
   - git init
8. **Initialize DVC:**
   - dvc init
9. **Make your first Git commit:**
   - git add .
   - git commit -m "initial project structure"
10. **Verify DVC initialized correctly:**
    - dvc status

11. **start writing actual code:**
    - **Open `config/config.yaml` in VS Code and paste this:**

```yaml
data:
  raw_path: "data/raw/dataset.csv"
  processed_path: "data/processed/features.csv"
  target_column: "species"
  test_size: 0.2
  random_state: 42

feature_engineering:
  numerical_strategy: "median"
  categorical_strategy: "most_frequent"
  scaling: "standard"

model_training:
  models:
    - name: "RandomForest"
      params:
        n_estimators: 100
        max_depth: 10
    - name: "XGBoost"
      params:
        n_estimators: 100
        learning_rate: 0.1
    - name: "LogisticRegression"
      params:
        max_iter: 1000

  metric: "f1_score"

mlflow:
  tracking_uri: "mlruns"
  experiment_name: "automl-experiment"

deployment:
  host: "0.0.0.0"
  port: 8000
  model_name: "best_model"
```

12. **Then verify it loads correctly:**

```
python -c "import yaml; cfg=yaml.safe_load(open('config/config.yaml'));
print('Config loaded!'); print('Target column:', cfg['data']['target_column'])"
```

You should see:

```
Config loaded!
Target column: target
```

13. **Now we write the first Python file —** `src/data_ingestion/ingest.py`

```
import pandas as pd
import yaml
import os
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)s |
%(message)s")
logger = logging.getLogger(__name__)


def load_config(path="config/config.yaml"):
    with open(path, "r") as f:
        return yaml.safe_load(f)


def ingest_data(config):
    raw_path = config["data"]["raw_path"]

    if not os.path.exists(raw_path):
        raise FileNotFoundError(f"File not found: {raw_path}")

    ext = os.path.splitext(raw_path)[-1].lower()

    if ext == ".csv":
        df = pd.read_csv(raw_path)
    elif ext == ".parquet":
        df = pd.read_parquet(raw_path)
```

```
    elif ext == ".json":
        df = pd.read_json(raw_path)
    else:
        raise ValueError(f"Unsupported format: {ext}")

    logger.info(f"Data loaded: {df.shape[0]} rows, {df.shape[1]} columns")

    target = config["data"]["target_column"]
    if target not in df.columns:
        raise ValueError(f"Target column '{target}' not found. Available:
{df.columns.tolist()}")

    logger.info(f"Target column '{target}' found OK")
    return df


if __name__ == "__main__":
    config = load_config()
    df = ingest_data(config)
    print(df.head())
```

14. **Now we need to make data source:**
    - **Make Folder name of**: Data/row
    - **Past your data file**.
    - **The correct folder structure is:**

```
automl-mlops-platform
└── data
    └── raw            ← folder
        └── dataset.csv  ← file goes HERE
```

15. **Now run this command:**
    - python src/data_ingestion/ingest.py

What we completed today

1. Created project folder automl-mlops-platform
2. Created virtual environment venv
3. Installed all dependencies with --prefer-binary
4. Created all project files and folders
5. Initialized Git and DVC
6. Wrote config/config.yaml
7. Wrote src/data_ingestion/ingest.py
8. Added Iris dataset to data/raw/dataset.csv

Before you sleep — save your work!

Run this in terminal:

```
git add .

git commit -m "completed data ingestion step"
```

DAY 2 Welcome back: `ingest.py` is working perfectly:

16. **Now we write the next file – [validate.py]**

- **Open `src/data_validation/validate.py` and paste this:**

```
import pandas as pd
import yaml
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)


def load_config(path="config/config.yaml"):
    with open(path, "r") as f:
        return yaml.safe_load(f)


class DataValidator:
    def __init__(self, config):
        self.target = config["data"]["target_column"]
        self.errors = []

    def check_target_exists(self, df):
        if self.target not in df.columns:
            self.errors.append(f"Target column '{self.target}' missing")

    def check_missing_values(self, df):
        for col in df.columns:
            missing = df[col].isna().mean()
            if missing > 0.4:
                self.errors.append(f"Column '{col}' has too many missing values:
{missing:.0%}")

    def check_minimum_rows(self, df):
        if len(df) < 50:
            self.errors.append(f"Too few rows: {len(df)} — need at least 50")

    def check_duplicates(self, df):
```

```
        dupes = df.duplicated().sum()
        if dupes > 0:
            logger.warning(f"Found {dupes} duplicate rows — will be dropped
later")

    def validate(self, df):
        self.check_target_exists(df)
        self.check_missing_values(df)
        self.check_minimum_rows(df)
        self.check_duplicates(df)

        if self.errors:
            for err in self.errors:
                logger.error(f"Validation failed: {err}")
            return False

        logger.info("All validation checks passed!")
        return True


if __name__ == "__main__":
    import pandas as pd
    config = load_config()
    df = pd.read_csv(config["data"]["raw_path"])
    validator = DataValidator(config)
    result = validator.validate(df)
    print("Validation passed!" if result else "Validation failed!")


—----------------------------------------------------------------------------------

validate.py is working perfectly!
INFO | All validation checks passed!
Validation passed!
```

17. then we write the next file – features.py.

- **Open `src/feature_engineering/features.py` and paste this:**

```
import pandas as pd
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, LabelEncoder,
OneHotEncoder
from sklearn.impute import SimpleImputer
import joblib
import yaml
import os
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)


def load_config(path="config/config.yaml"):
    with open(path, "r") as f:
        return yaml.safe_load(f)


def engineer_features(df, config):
    target = config["data"]["target_column"]
    os.makedirs("artifacts", exist_ok=True)

    # Drop duplicates
    df = df.drop_duplicates()

    # Drop Id column if exists
    if "Id" in df.columns:
        df = df.drop(columns=["Id"])
        logger.info("Dropped 'Id' column")

    # Split features and target
    X = df.drop(columns=[target])
    y = df[target]

    # Encode target labels to numbers
    le = LabelEncoder()
```

```
    y_encoded = le.fit_transform(y)
    joblib.dump(le, "artifacts/label_encoder.pkl")
    logger.info(f"Target classes: {le.classes_}")

    # Find numerical and categorical columns
    num_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
    cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
    logger.info(f"Numerical columns: {num_cols}")
    logger.info(f"Categorical columns: {cat_cols}")

    # Build preprocessing pipeline
    num_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    transformers = [("num", num_pipeline, num_cols)]

    if cat_cols:
        cat_pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OneHotEncoder(handle_unknown="ignore",
sparse_output=False)),
        ])
        transformers.append(("cat", cat_pipeline, cat_cols))

    preprocessor = ColumnTransformer(transformers)

    # Transform features
    X_transformed = preprocessor.fit_transform(X)
    joblib.dump(preprocessor, "artifacts/preprocessor.pkl")
    logger.info(f"Features shape after transformation:
{X_transformed.shape}")
    logger.info("Preprocessor saved to artifacts/preprocessor.pkl")

    return X_transformed, y_encoded, preprocessor


if __name__ == "__main__":
    config = load_config()
```

```
df = pd.read_csv(config["data"]["raw_path"])
X, y, preprocessor = engineer_features(df, config)
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"Sample y values: {y[:5]}")
```

- python src/feature_engineering/features.py

  You should see: ``

  ` INFO | Dropped 'Id' column

  INFO | Target classes: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

  INFO | Numerical columns: [...]

  X shape: (150, 4)

  y shape: (150,)

18. then write the next file – train.py.

- **Open `src/model_training/train.py` and paste this:**

```
import mlflow
import mlflow.sklearn
import pandas as pd
import joblib
import yaml
import logging
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import f1_score, accuracy_score
from xgboost import XGBClassifier
import sys
import os
sys.path.insert(0, os.path.abspath("."))
```

```
logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)


def load_config(path="config/config.yaml"):
    with open(path, "r") as f:
        return yaml.safe_load(f)


MODEL_MAP = {
    "RandomForest": RandomForestClassifier,
    "XGBoost": XGBClassifier,
    "LogisticRegression": LogisticRegression,
    "SVC": SVC,
}


def train_all_models(X, y, config):
    mlflow_cfg = config["mlflow"]
    mlflow.set_tracking_uri(mlflow_cfg["tracking_uri"])
    mlflow.set_experiment(mlflow_cfg["experiment_name"])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=config["data"]["test_size"],
        random_state=config["data"]["random_state"]
    )

    best_score = -1
    best_model = None
    best_model_name = None

    for model_cfg in config["model_training"]["models"]:
        name = model_cfg["name"]
        params = model_cfg.get("params", {})

        with mlflow.start_run(run_name=name):
            logger.info(f"Training {name}...")
            model = MODEL_MAP[name](**params)
```

```
            model.fit(X_train, y_train)

            y_pred = model.predict(X_test)
            f1 = f1_score(y_test, y_pred, average="weighted")
            acc = accuracy_score(y_test, y_pred)

            mlflow.log_param("model_name", name)
            mlflow.log_params(params)
            mlflow.log_metric("f1_score", f1)
            mlflow.log_metric("accuracy", acc)
            mlflow.sklearn.log_model(model, "model")

            logger.info(f"{name} — F1: {f1:.4f} | Accuracy: {acc:.4f}")

            if f1 > best_score:
                best_score = f1
                best_model = model
                best_model_name = name

    os.makedirs("artifacts", exist_ok=True)
    joblib.dump(best_model, "artifacts/best_model.pkl")
    logger.info(f"Best model: {best_model_name} with F1={best_score:.4f}")
    logger.info("Best model saved to artifacts/best_model.pkl")

    return best_model, best_model_name, best_score


if __name__ == "__main__":
    from src.feature_engineering.features import engineer_features
    config = load_config()
    df = pd.read_csv(config["data"]["raw_path"])
    X, y, _ = engineer_features(df, config)
    best_model, best_name, best_score = train_all_models(X, y, config)
    print(f"\nWinner: {best_name} with F1 = {best_score:.4f}")
```

- **python src/model_training/train.py**

All 3 models trained successfully! 🎉

```
RandomForest  — F1: 1.0000 | Accuracy: 1.0000

XGBoost       — F1: 1.0000 | Accuracy: 1.0000

LogisticRegression — F1: 1.0000 | Accuracy: 1.0000

Winner: RandomForest with F1 = 1.0000
```

- **MLflow is also tracking everything — you can see `mlruns` folder appeared in your project!**
- Now let's open the MLflow UI to see all experiments visually:

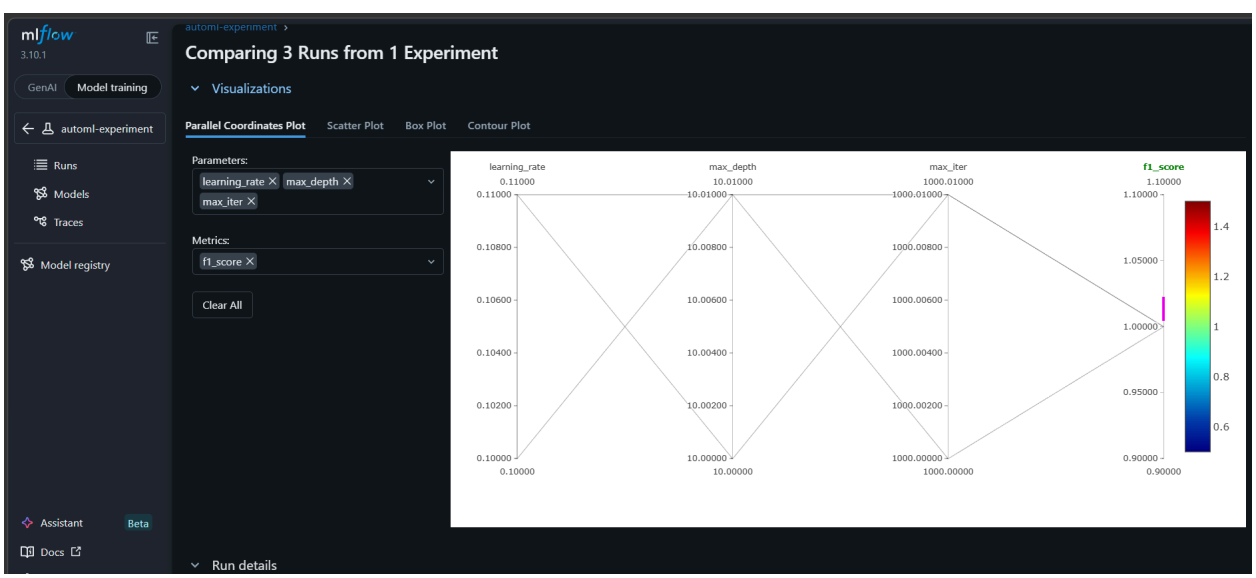
mlflow ui

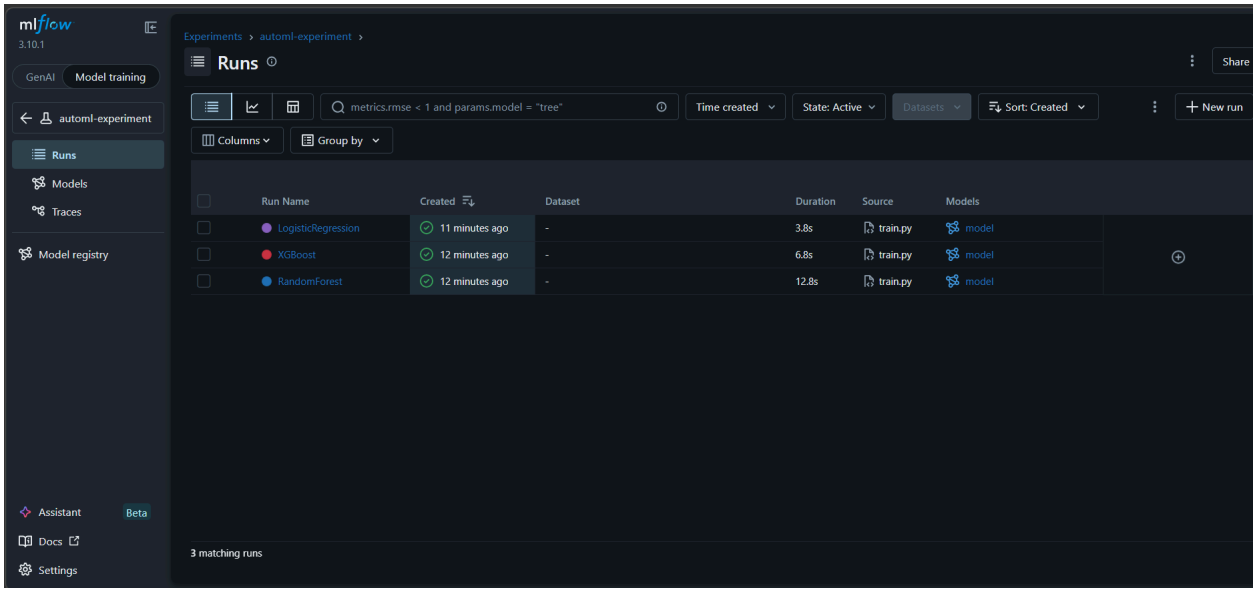
```

```

Then open your browser and go to:

```

```

http://127.0.0.1:5000





- **This is amazing! 🎉 MLflow UI is working perfectly**
- **Now press `Ctrl + C` in terminal to stop MLflow**

19.write the last important file – [api.py](api.py).

- **Open `src/deployment/api.py` and paste this:**

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import PlainTextResponse
import joblib
import pandas as pd
import time
import logging
import os

logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)

app = FastAPI(title="AutoML Prediction API", version="1.0")
```

```
# Prometheus metrics
PREDICTION_COUNTER = Counter("predictions_total", "Total predictions
made")
PREDICTION_LATENCY = Histogram("prediction_latency_seconds",
"Prediction latency")
ERROR_COUNTER = Counter("prediction_errors_total", "Total errors")

model = None
preprocessor = None
label_encoder = None

@app.on_event("startup")
def load_artifacts():
    global model, preprocessor, label_encoder
    model = joblib.load("artifacts/best_model.pkl")
    preprocessor = joblib.load("artifacts/preprocessor.pkl")
    label_encoder = joblib.load("artifacts/label_encoder.pkl")
    logger.info("All artifacts loaded successfully!")

class PredictRequest(BaseModel):
    SepalLengthCm: float
    SepalWidthCm: float
    PetalLengthCm: float
    PetalWidthCm: float

class PredictResponse(BaseModel):
    prediction: str
    confidence: float

@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": model is not None}

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    start = time.time()
    try:
        df = pd.DataFrame([request.dict()])
        X = preprocessor.transform(df)
```

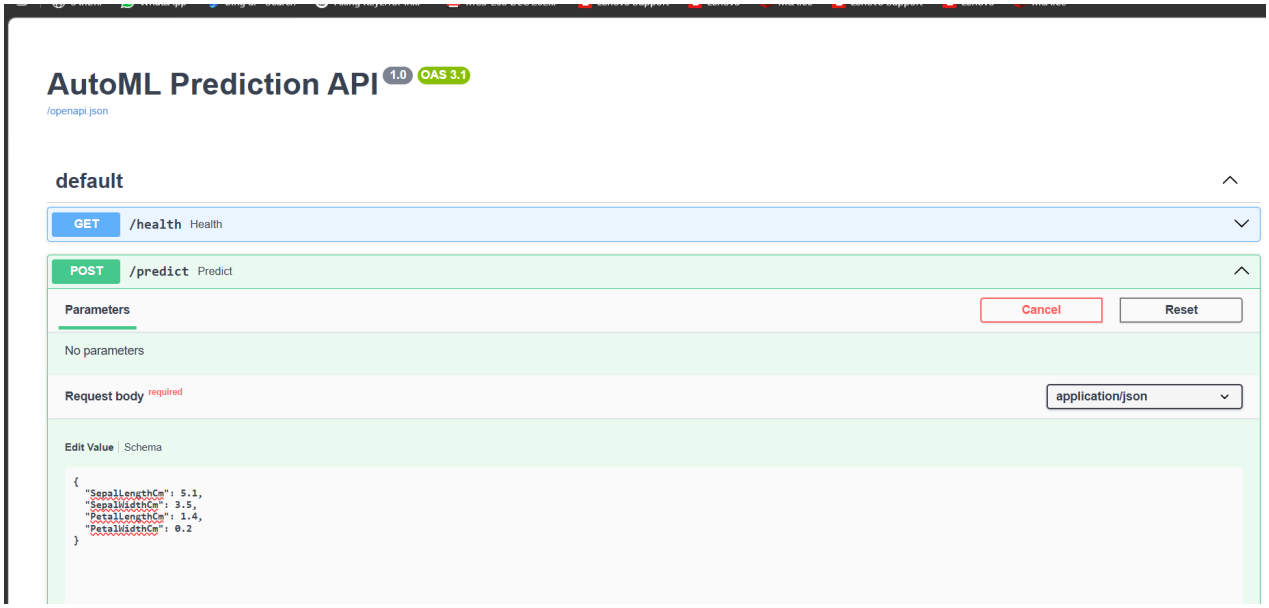
```
        pred = model.predict(X)[0]
        proba = model.predict_proba(X)[0]
        confidence = float(max(proba))
        species = label_encoder.inverse_transform([pred])[0]

        PREDICTION_COUNTER.inc()
        PREDICTION_LATENCY.observe(time.time() - start)

        return PredictResponse(prediction=species, confidence=confidence)

    except Exception as e:
        ERROR_COUNTER.inc()
        logger.error(f"Prediction failed: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/metrics", response_class=PlainTextResponse)
def metrics():
    return generate_latest()
```

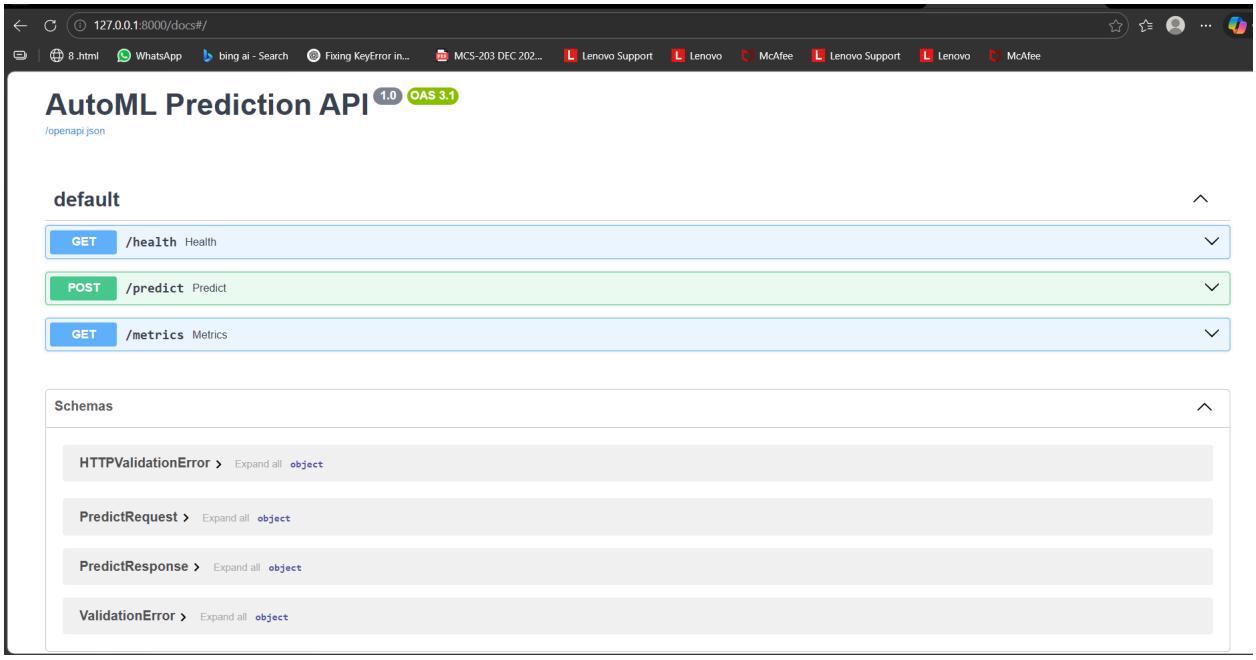
—----------

```
uvicorn src.deployment.api:app --reload --port 8000
```

Then open browser and go to:

```
http://127.0.0.1:8000/docs
```


AutoML Prediction API
1.0
OAS 3.1
/openapi.json
default
GET /health Health
POST /predict Predict
Parameters
Cancel
Reset
No parameters
Request body required
application/json
Edit Value | Schema
{
"SepalLengthCm": 5.1,
"SepalWidthCm": 3.5,
"PetalLengthCm": 1.4,
"PetalWidthCm": 0.2
}


127.0.0.1:8000/docs#/
8 .html
WhatsApp
bing ai - Search
Fixing KeyError in...
MCS-203 DEC 202...
Lenovo Support
Lenovo
McAfee
Lenovo Support
Lenovo
McAfee
AutoML Prediction API
1.0
OAS 3.1
/openapi.json
default
GET /health Health
POST /predict Predict
GET /metrics Metrics
Schemas
HTTPValidationError Expand all object
PredictRequest Expand all object
PredictResponse Expand all object
ValidationError Expand all object

20. Let's do it step by step! Let's finish the core files first, then add the bonus features.

- **Open `src/model_evaluation/evaluate.py` and paste:**

```
import joblib
import json
import os
import yaml
import logging
import pandas as pd
from sklearn.metrics import f1_score, accuracy_score,
classification_report
from sklearn.model_selection import train_test_split

logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)


def load_config(path="config/config.yaml"):
    with open(path, "r") as f:
        return yaml.safe_load(f)


def evaluate_model(config):
    os.makedirs("artifacts", exist_ok=True)

    model = joblib.load("artifacts/best_model.pkl")
    preprocessor = joblib.load("artifacts/preprocessor.pkl")
    label_encoder = joblib.load("artifacts/label_encoder.pkl")

    df = pd.read_csv(config["data"]["raw_path"])
    if "Id" in df.columns:
        df = df.drop(columns=["Id"])

    target = config["data"]["target_column"]
    X = df.drop(columns=[target])
    y = label_encoder.transform(df[target])

    X_transformed = preprocessor.transform(X)
```

```
    _, X_test, _, y_test = train_test_split(
        X_transformed, y,
        test_size=config["data"]["test_size"],
        random_state=config["data"]["random_state"]
    )

    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average="weighted")
    acc = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred,
                target_names=label_encoder.classes_, output_dict=True)

    metrics = {
        "f1_score": round(f1, 4),
        "accuracy": round(acc, 4),
        "model_type": type(model).__name__,
        "classification_report": report
    }

    with open("artifacts/metrics.json", "w") as f:
        json.dump(metrics, f, indent=2)

    logger.info(f"F1 Score : {f1:.4f}")
    logger.info(f"Accuracy : {acc:.4f}")
    logger.info("Metrics saved to artifacts/metrics.json")
    return metrics


if __name__ == "__main__":
    config = load_config()
    evaluate_model(config)
```

---

python src/model_evaluation/evaluate.py

21. **Training_pipeline.py**
    - **Open pipeline/training_pipeline.py and paste:**

```
import sys
import os
sys.path.insert(0, os.path.abspath("."))

import yaml
import logging
import pandas as pd

from src.data_ingestion.ingest import load_config, ingest_data
from src.data_validation.validate import DataValidator
from src.feature_engineering.features import engineer_features
from src.model_training.train import train_all_models
from src.model_evaluation.evaluate import evaluate_model

logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)s |
%(message)s")
logger = logging.getLogger(__name__)


def run_pipeline():
    logger.info("="*50)
    logger.info("   AUTOML PIPELINE STARTING")
    logger.info("="*50)

    # Step 1 - Load config
    config = load_config()

    # Step 2 - Ingest data
    logger.info("STEP 1: Data Ingestion")
    df = ingest_data(config)

    # Step 3 - Validate data
    logger.info("STEP 2: Data Validation")
    validator = DataValidator(config)
    if config["data"]["target_column"] not in df.columns:
        config["data"]["target_column"] = df.columns[-1]
        logger.info(f"Auto-detected target column: {df.columns[-1]}")
```

```
    validator = DataValidator(config)
    if not validator.validate(df):
        raise ValueError("Data validation failed! Pipeline stopped.")
    # Step 4 - Feature engineering
    logger.info("STEP 3: Feature Engineering")
    X, y, preprocessor = engineer_features(df, config)

    # Step 5 - Train models
    logger.info("STEP 4: Model Training")
    best_model, best_name, best_score = train_all_models(X, y, config)

    # Step 6 - Evaluate
    logger.info("STEP 5: Model Evaluation")
    metrics = evaluate_model(config)

    logger.info("="*50)
    logger.info(f"PIPELINE COMPLETE!")
    logger.info(f"Best Model : {best_name}")
    logger.info(f"F1 Score   : {best_score:.4f}")
    logger.info(f"API ready  : run 'uvicorn src.deployment.api:app --port 8000'")
    logger.info("="*50)


if __name__ == "__main__":
    run_pipeline()
```

—----------

```
python pipeline/training_pipeline.py
```

22. Step 3 — `Dockerfile`
    - `Open Dockerfile and paste:`

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --prefer-binary

COPY src/ src/
COPY config/ config/
COPY artifacts/ artifacts/
COPY pipeline/ pipeline/

EXPOSE 8000

CMD ["uvicorn", "src.deployment.api:app", "--host", "0.0.0.0", "--port",
"8000"]
```

23 — `.github/workflows/ci.yaml`

Open `.github/workflows/ci.yaml` and paste:

```yaml
name: AutoML CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  run-pipeline:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: pip install -r requirements.txt --prefer-binary

      - name: Run full pipeline
        run: python pipeline/training_pipeline.py

      - name: Start API and health check
        run: |
          uvicorn src.deployment.api:app --port 8000 &
          sleep 5
          curl -f http://localhost:8000/health

      - name: Build Docker image
        run: docker build -t automl-platform .
```

24. Bonus Feature: Upload UI + Dashboard

```
from fastapi import FastAPI, HTTPException, UploadFile, File
from fastapi.responses import PlainTextResponse, HTMLResponse
from pydantic import BaseModel
from prometheus_client import Counter, Histogram, generate_latest
import joblib
import pandas as pd
import time
import logging
import os
import json
import shutil
import sys

logging.basicConfig(level=logging.INFO, format="%(asctime)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)

app = FastAPI(title="AutoML Prediction API", version="2.0")

PREDICTION_COUNTER = Counter("predictions_total", "Total predictions
made")
PREDICTION_LATENCY = Histogram("prediction_latency_seconds",
"Prediction latency")
ERROR_COUNTER = Counter("prediction_errors_total", "Total errors")

model = None
preprocessor = None
label_encoder = None


@app.on_event("startup")
def load_artifacts():
    global model, preprocessor, label_encoder
    model = joblib.load("artifacts/best_model.pkl")
    preprocessor = joblib.load("artifacts/preprocessor.pkl")
    label_encoder = joblib.load("artifacts/label_encoder.pkl")
    logger.info("All artifacts loaded!")
```

```
class PredictRequest(BaseModel):
    SepalLengthCm: float
    SepalWidthCm: float
    PetalLengthCm: float
    PetalWidthCm: float


class PredictResponse(BaseModel):
    prediction: str
    confidence: float


@app.get("/", response_class=HTMLResponse)
def home():
    return dashboard()


@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    start = time.time()
    try:
        df = pd.DataFrame([request.dict()])
        X = preprocessor.transform(df)
        pred = model.predict(X)[0]
        proba = model.predict_proba(X)[0]
        confidence = float(max(proba))
        species = label_encoder.inverse_transform([pred])[0]
        PREDICTION_COUNTER.inc()
        PREDICTION_LATENCY.observe(time.time() - start)
        return PredictResponse(prediction=species, confidence=confidence)
    except Exception as e:
        ERROR_COUNTER.inc()
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/upload")
async def upload_dataset(file: UploadFile = File(...)):
    import subprocess
```

```
    os.makedirs("data/raw", exist_ok=True)
    save_path = f"data/raw/dataset.csv"

    with open(save_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    df = pd.read_csv(save_path)

    # Retrain pipeline automatically
    try:
        subprocess.run(
            [sys.executable, "pipeline/training_pipeline.py"],
            check=True
            )
        # Reload model artifacts
        global model, preprocessor, label_encoder
        model = joblib.load("artifacts/best_model.pkl")
        preprocessor = joblib.load("artifacts/preprocessor.pkl")
        label_encoder = joblib.load("artifacts/label_encoder.pkl")
        retrained = True
        message = "File uploaded and model retrained successfully!"
    except Exception as e:
        retrained = False
        message = f"File uploaded but retraining failed: {str(e)}"

    return {
        "message": message,
        "filename": file.filename,
        "rows": len(df),
        "columns": df.columns.tolist(),
        "retrained": retrained
    }


@app.get("/dashboard", response_class=HTMLResponse)
def dashboard():
    metrics = {}
    if os.path.exists("artifacts/metrics.json"):
        with open("artifacts/metrics.json") as f:
            metrics = json.load(f)
```

```
report = metrics.get("classification_report", {})
rows = ""
for cls, vals in report.items():
    if isinstance(vals, dict):
        rows += f"""
        <tr>
            <td>{cls}</td>
            <td>{vals.get('precision', 0):.2f}</td>
            <td>{vals.get('recall', 0):.2f}</td>
            <td>{vals.get('f1-score', 0):.2f}</td>
            <td>{int(vals.get('support', 0))}</td>
        </tr>"""

html = f"""
<!DOCTYPE html>
<html>
<head>
    <title>AutoML Dashboard</title>
    <style>
        body {{ font-family: Arial, sans-serif; margin: 40px;
               background: #0f172a; color: #e2e8f0; }}
        h1 {{ color: #38bdf8; }}
        h2 {{ color: #94a3b8; border-bottom: 1px solid #334155;
              padding-bottom: 8px; }}
        .card {{ background: #1e293b; border-radius: 12px;
                 padding: 24px; margin: 20px 0; }}
        .metric {{ display: inline-block; background: #0f172a;
                   border-radius: 8px; padding: 16px 32px;
                   margin: 8px; text-align: center; }}
        .metric .value {{ font-size: 2em; color: #38bdf8;
                          font-weight: bold; }}
        .metric .label {{ color: #94a3b8; font-size: 0.9em; }}
        table {{ width: 100%; border-collapse: collapse; }}
        th {{ background: #334155; padding: 12px; text-align: left; }}
        td {{ padding: 10px 12px; border-bottom: 1px solid #1e293b; }}
        tr:hover td {{ background: #1e293b; }}
        .upload-form {{ background: #1e293b; border-radius: 12px;
                        padding: 24px; margin: 20px 0; }}
        input[type=file] {{ color: #e2e8f0; margin: 10px 0; }}
```

```
        button {{ background: #38bdf8; color: #0f172a; border: none;
                  padding: 10px 24px; border-radius: 6px;
                  cursor: pointer; font-weight: bold; }}
        button:hover {{ background: #0ea5e9; }}
        #result {{ margin-top: 12px; color: #4ade80; }}
    </style>
</head>
<body>
    <h1>AutoML Platform Dashboard</h1>

    <div class="card">
        <h2>Model Performance</h2>
        <div class="metric">
            <div class="value">{metrics.get('accuracy', 0):.2%}</div>
            <div class="label">Accuracy</div>
        </div>
        <div class="metric">
            <div class="value">{metrics.get('f1_score', 0):.2%}</div>
            <div class="label">F1 Score</div>
        </div>
        <div class="metric">
            <div class="value">{metrics.get('model_type', 'N/A')}</div>
            <div class="label">Best Model</div>
        </div>
    </div>

    <div class="card">
        <h2>Classification Report</h2>
        <table>
            <tr>
                <th>Class</th>
                <th>Precision</th>
                <th>Recall</th>
                <th>F1 Score</th>
                <th>Support</th>
            </tr>
            {rows}
        </table>
    </div>
```

```
        <div class="upload-form">
            <h2>Upload New Dataset</h2>
            <input type="file" id="fileInput" accept=".csv"/>
            <br/>
            <button onclick="uploadFile()">Upload Dataset</button>
            <div id="result"></div>
        </div>

        <script>
        async function uploadFile() {{
            const file = document.getElementById('fileInput').files[0];
            if (!file) {{ alert('Please select a file!'); return; }}

            document.getElementById('result').innerHTML = 'Uploading and
retraining... please wait...';
            document.getElementById('result').style.color = '#facc15';

            const form = new FormData();
            form.append('file', file);

            const res = await fetch('/upload', {{method:'POST', body:form}});
            const data = await res.json();

            if (data.retrained) {{
                document.getElementById('result').innerHTML =
                    'Done! Rows: ' + data.rows +
                    ' | Model retrained! Refreshing...';
                document.getElementById('result').style.color = '#4ade80';
                setTimeout(() => location.reload(), 2000);
            }} else {{
                document.getElementById('result').innerHTML = data.message;
                document.getElementById('result').style.color = '#f87171';
            }}
        }}
        </script>
    </body>
    </html>
    """
    return HTMLResponse(content=html)
```

```
@app.get("/metrics", response_class=PlainTextResponse)
def metrics():
    return generate_latest()
```

```
uvicorn src.deployment.api:app --reload --port 8000
```

→ **Almost complete! Let me show you what is done and what is still remaining:**

Done ✅

ingest.py	✅ **loads any dataset**
validate.py	✅ **checks data quality**
features.py	✅ **engineers features**
train.py	✅ **trains 3 ML models**
evaluate.py	✅ **saves metrics**
training_pipeline	✅ **one command runs all**
api.py	✅ **live prediction API**
dashboard	✅ **model comparison UI**
upload UI	✅ **upload new datasets**
MLflow	✅ **experiment tracking**

Still remaining ⏳

Dockerfile	⏳ **not tested yet**
ci.yaml	⏳ **needs GitHub repo**
README.md	⏳ **needs to be written**
git commit	⏳ **save all work**

Do these 3 things to finish completely:

Step 1 — Save everything to Git:

- git add .
- git commit -m "complete automl platform v1.0"

Step 2 — Push to GitHub (create a free account at github.com first):

- git remote add origin https://github.com/YOURNAME/automl-mlops-platform.git
- git push -u origin main

Step 3 — Test pipeline one final time:

- python pipeline/training_pipeline.py

Built an end-to-end AutoML MLOps platform with automated training pipeline, REST API deployment, experiment tracking with MLflow, data versioning with DVC, and real-time monitoring dashboard.